Lab Handout 3-F

Python Collection Script and Campaigns C1 and C2

Capstone — Module 3: System Integration and Data Collection

1 Objective

Implement the host-side Python data collection script that receives the Threat's 43-byte forwarding frames passed through the Field-Programmable Gate Array (FPGA)'s USB-UART bridge, parses the embedded radio packet fields, attaches ground-truth labels from a campaign configuration file, and writes labeled records to a CSV file. Then execute the first two single-variable campaigns: a TX power sweep (C1) and a spreading factor sweep (C2).

2 Prerequisites

- Lab Handout 3-E complete (production bitstream ready, performance envelope documented).
- Python 3.10+ on the host PC with `pyserial` $\geq$ `3.5`, `pandas` $\geq$ `2.0`, and `matplotlib` installed.
- FPGA USB-UART passthrough verified: `ingest_top.vhd` routes `uart_rx_pin` directly to `UART_RXD_OUT`, so the host PC sees the raw Threat forwarding frames on the Nexys A7's onboard USB-UART bridge at 115200 8N1.

3 Forwarding Frame Format

The Threat emits a 43-byte forwarding frame over USART3 TX for every intercepted radio packet. The FPGA passes this through unchanged to the host USB-UART. The collection script parses these frames directly — there is no separate feature-export path on the FPGA.

Field	Offset	Size	Description
Start Delimiter	0	1 B	`0x7E`
Length	1	1 B	`0x29` (41 decimal; bytes covered by CRC)
`rx_timestamp_ms`	2–5	4 B	Threat `HAL_GetTick()` at `RxDone`, little-endian uint32
`rssi_dbm`	6	1 B	Signed int8, raw negative dBm (e.g. `0xCE` = −50 dBm)
`snr_q025`	7	1 B	Signed int8, 0.25 dB units (divide by 4 for dB)
`freq_error_hz`	8–9	2 B	Signed int16, little-endian, Hz
Radio Payload	10–41	32 B	Raw intercepted LoRa packet (see embedded fields below)
CRC-8/ITU	42	1 B	Polynomial `0x07`, init `0x00`, covers bytes 1–41

Embedded fields inside the 32-byte radio payload (offsets relative to byte 10)

Field	Frame Offset	Size	Description
`beacon_id`	12	1 B	Sender ID (`0x01`=Target A, `0x02`=Target B)
`seq_num`	13–14	2 B	Little-endian uint16 sequence counter
`payload_len`	15	1 B	Inner radio payload length
`mod_code`	16	1 B	Bits[1:0] = packet type (`00`=DATA, `01`=ACK); bits[7:2] = modulation kind
`spread_factor`	17	1 B	SF7–SF10
`tx_timestamp_ms`	19–22	4 B	Originator `HAL_GetTick()` at TX, little-endian uint32

4 Procedure

4.1 Step 1 — Campaign configuration JSON

Each campaign run is described by a JSON configuration file that the collection script reads at startup. The configuration provides the ground-truth labels that are attached to each received feature vector.

```
{
    "campaign_id":          "C1",
    "run_id":               "C1_pwr02_2026-04-20T14:00:00Z",
    "sweep_variable":       "tx_power",
    "sweep_value":          2,
    "fixed_sf":             7,
    "fixed_mod":            "lora",
    "fixed_mod_kind":       1,
    "fixed_distance_m":     3.0,
    "num_active_stations": 1,
    "expected_pkt_types":   [0],
    "target_packets":       200,
    "notes":                "TX power sweep, Target A only (DATA only)",
    "description":          "C1 TX Power Sweep -- +2 dBm, SF7, 200 packets,
        Target A"
}
```

Listing 1: Example campaign configuration file `campaign_c1_txpwr_2.json`.

campaign_id vs run_id

`campaign_id` identifies the campaign *family* (C1, C2, . . . , C5). `run_id` identifies one specific collection execution (a single invocation of `collect.py` at one sweep step, with an ISO-8601 timestamp to ensure uniqueness across reruns). Every row written to CSV must carry both. Downstream scripts (LH3-H merge, LH4-A/G splits) key uniqueness and split-leakage checks on `run_id`, not on `campaign_id`. Subcampaign labels (e.g. C5a/C5b/C5c in LH3-G) live in

`campaign_id`; a subcampaign can contain multiple `run_id`s.

Modulation-Kind and Packet-Type Canonical Mappings

`fixed_mod_kind` uses the Module 1/Module 2 canonical integer codes: **1=LoRa, 2=FSK, 3=OOK**. Store this integer directly in the CSV as `mod_kind_label` so that Module 4 ingest does not need a string-to-int conversion table. The feature vector's `mod_kind` field at offset 1 uses the same mapping with the low 2 bits cleared (the LH2-F extractor already does this). Do *not* introduce a 0/1 alias.

The packet-type field uses **0=DATA, 1=ACK**. The `expected_pkt_types` list in the JSON tells the script which `pkt_type` values are valid for this run; rows with a `pkt_type` outside the list are flagged as cross-domain anomalies but still recorded. (PAUSE frames originate at the Threat and are transmitted over-the-air to silence Targets; they are not received back through the Threat's intercept path, so they never appear in the dataset.)

One JSON file exists per sweep step. The `Code/Tools/CampaignCollect/campaigns/` directory already contains all 11 files: `dry_run.json`, four C1 TX-power files (`campaign_c1_txpwr_2.json` through `campaign_c1_txpwr_20.json`), and six C2 spreading-factor files (`campaign_c2_sf7.json` through `campaign_c2_sf12.json`). Each file includes a `description` field that `collect.py` displays in the selection menu. Output CSVs are written under `Code/Tools/CampaignCollect/logs/<config_stem>/`.

4.2 Step 2 — Refactor Target firmware for campaign sweeps

In Lab 2 you wrote `lora_init()` with the spreading factor and TX power baked in as raw register bytes. Before running any campaign sweep those values need to change repeatedly, so refactor both Target projects now to expose them as named defines at the top of the file. This is a one-time change; all subsequent steps only require editing two lines.

1. In VS Code, open the `Nucleo_RFM95_TargetA` folder and navigate to `Core/Src/main.c`. In the `/* USER CODE BEGIN PD */` block, add the following define block after your existing beacon/role defines:

```
/* Campaign-configurable radio parameters.
 * Change LORA_SF and LORA_TX_PWR_DBM between runs.
 * Do not edit the derived macros or any register addresses. */
#define LORA_SF           7
#define LORA_TX_PWR_DBM   14

#define REG_MODEM_CONFIG2_VAL  ((LORA_SF << 4) | 0x04)
#define REG_PA_CONFIG_VAL      (0x80 | ((LORA_TX_PWR_DBM - 2) & 0x0F))
```

2. In `lora_init()`, find the two lines where you wrote the spreading factor and TX power registers and replace the raw byte literals with the new macros:

```
rfm_write_reg(0x1E, REG_MODEM_CONFIG2_VAL);  /* was 0x74 (SF7) */
rfm_write_reg(0x09, REG_PA_CONFIG_VAL);       /* was 0x8C (+14 dBm) */
```

3. Update the `printf` at the end of `lora_init()` to confirm the active configuration on the debug terminal:

```
printf("lora_init done: SF=%d TxPwr=%d dBm\r\n",
       LORA_SF, LORA_TX_PWR_DBM);
```

4. Repeat the identical changes in `Nucleo_RFM95_TargetB/Core/Src/main.c`.

5. Press **F7** in each VS Code window to build, then click **Start Debugging** (or press **F5**) to flash and run both boards. Confirm both debug terminals print `lora_init done:  SF=7 TxPwr=14 dBm` before continuing.

4.3 Step 3 — Implement the collection script

Create `collect.py`. The script must: open the serial port, synchronize on the `0x7E` start delimiter, read each 43-byte forwarding frame, verify the CRC-8/ITU over bytes 1–41, extract the radio-packet fields from the embedded payload, attach labels from the configuration JSON, and append one row to a CSV file.

```
import serial
import csv
import json
import sys
from datetime import datetime

BAUD = 115200
```

```python
FRAME_LEN = 43
START = 0x7E
LENGTH = 0x29  # bytes 1..41 covered by CRC

def crc8_itu(data, poly=0x07, init=0x00):
    crc = init
    for b in data:
        crc ^= b
        for _ in range(8):
            crc = ((crc << 1) ^ poly) & 0xFF if (crc & 0x80) \
                  else (crc << 1) & 0xFF
    return crc

def parse_forwarding_frame(buf43):
    """Unpack a 43-byte Threat forwarding frame into a dict."""
    rx_ts_ms      = int.from_bytes(buf43[2:6],  "little")
    rssi_dbm      = int.from_bytes(buf43[6:7],  "little", signed=True)
    snr_q025      = int.from_bytes(buf43[7:8],  "little", signed=True)
    freq_err_hz   = int.from_bytes(buf43[8:10], "little", signed=True)
    payload       = buf43[10:42]   # 32-byte raw radio packet

    beacon_id     = payload[2]
    seq_num       = int.from_bytes(payload[3:5], "little")
    payload_len   = payload[5]
    mod_code      = payload[6]
    pkt_type      = mod_code & 0x03           # 0=DATA, 1=ACK
    mod_kind      = (mod_code >> 2) & 0x3F    # upper 6 bits
    spread_factor = payload[7]
    tx_ts_ms      = int.from_bytes(payload[9:13], "little")

    return {
        "rx_timestamp_ms":   rx_ts_ms,
        "rssi_dbm":          rssi_dbm,
        "snr_db":            snr_q025 / 4.0,
        "freq_error_hz":     freq_err_hz,
        "beacon_id":         beacon_id,
        "seq_num":           seq_num,
        "payload_len":       payload_len,
        "mod_kind":          mod_kind,
        "spread_factor":     spread_factor,
        "pkt_type":          pkt_type,
        "tx_timestamp_ms":   tx_ts_ms,
    }

def collect(port, config_path, out_csv):
    with open(config_path) as f:
        cfg = json.load(f)
    target = cfg["target_packets"]
    received = 0
    last_rx_ts = None
    with serial.Serial(port, BAUD, timeout=2.0) as ser, \
         open(out_csv, "a", newline="") as csvf:
        writer = csv.DictWriter(csvf, fieldnames=CSV_COLUMNS)
        if csvf.tell() == 0:
```

```
            writer.writeheader()
        print(f"Collecting {target} packets for {cfg['campaign_id']} "
              f"sweep={cfg['sweep_value']}...")
        while received < target:
            b = ser.read(1)
            if not b or b[0] != START:
                continue
            length = ser.read(1)
            if not length or length[0] != LENGTH:
                continue   # bad length, resync
            rest = ser.read(FRAME_LEN - 2)
            if len(rest) < FRAME_LEN - 2:
                continue
            buf = bytes([START, LENGTH]) + rest
            expected_crc = crc8_itu(buf[1:42])
            if expected_crc != buf[42]:
                print(f"  [WARN] CRC mismatch: got {buf[42]:#04x}, "
                      f"expected {expected_crc:#04x}")
                continue
            fv = parse_forwarding_frame(buf)
            inter_arrival_ms = (fv["rx_timestamp_ms"] - last_rx_ts) \
                               if last_rx_ts is not None else 0
            last_rx_ts = fv["rx_timestamp_ms"]
            # Mismatch check between self-reported and config labels
            if fv["spread_factor"] != cfg["fixed_sf"]:
                print(f"  [WARN] SF mismatch: reported {fv['spread_factor'], "
                      f"config {cfg['fixed_sf']}")
            if fv["pkt_type"] not in cfg.get("expected_pkt_types", [0,1,2]):
                print(f"  [WARN] unexpected pkt_type={fv['pkt_type']} "
                      f"(campaign expected {cfg.get('expected_pkt_types')})")
            row = {**fv,
                   "inter_arrival_ms":  inter_arrival_ms,
                   "campaign_id":       cfg["campaign_id"],
                   "run_id":            cfg["run_id"],
                   "timestamp_host":    datetime.utcnow().isoformat(),
                   "distance_m":        cfg["fixed_distance_m"],
                   "label_sf":          cfg["fixed_sf"],
                   "label_mod":         cfg["fixed_mod"],
                   "mod_kind_label":    cfg["fixed_mod_kind"],
                   "label_beacon":      fv["beacon_id"],
                   "label_pkt":         fv["pkt_type"],
                   }
            writer.writerow(row)
            received += 1
            if received % 50 == 0:
                print(f"  {received}/{target} packets collected.")
    print(f"Done. {received} packets written to {out_csv}.")

CSV_COLUMNS = [
    "campaign_id", "run_id", "timestamp_host",
    "rx_timestamp_ms", "tx_timestamp_ms", "inter_arrival_ms",
```

```python
    "beacon_id", "seq_num", "pkt_type", "mod_kind", "spread_factor",
    "rssi_dbm", "snr_db", "freq_error_hz", "payload_len",
    "distance_m", "label_sf", "label_mod", "mod_kind_label",
    "label_beacon", "label_pkt",
]

def _discover_configs():
    from pathlib import Path
    campaigns_dir = Path(__file__).parent / "campaigns"
    entries = []
    for p in sorted(campaigns_dir.glob("*.json")):
        try:
            with open(p) as f:
                cfg = json.load(f)
            entries.append((p, cfg.get("description",
                                       cfg.get("notes", p.stem))))
        except Exception:
            pass
    return entries

def _menu():
    configs = _discover_configs()
    if not configs:
        print("No JSON config files found in script directory.")
        sys.exit(1)
    print("\n=== Campaign Collect ===")
    print("Available configurations:\n")
    for i, (path, desc) in enumerate(configs, 1):
        print(f"  {i:2d}.  {desc}")
        print(f"       ({path.name})")
    print()
    while True:
        try:
            idx = int(input("Select config number: ").strip()) - 1
            if 0 <= idx < len(configs):
                config_path = configs[idx][0]
                break
            print(f"  Enter a number between 1 and {len(configs)}.")
        except (ValueError, EOFError):
            print("  Invalid input.")
    port = input("COM port (e.g. COM3): ").strip()
    if not port:
        print("No port entered.")
        sys.exit(1)
    default_csv = f"results_{config_path.stem}.csv"
    out_csv = input(f"Output CSV [{default_csv}]: ").strip() \
              or default_csv
    print()
    collect(port, str(config_path), out_csv)

if __name__ == "__main__":
    _menu()
```

Listing 2: Collection script skeleton (`collect.py`).

Label Integrity

The `label_*` columns are derived from the campaign configuration (what you set the originating station to), *not* from the on-air fields (what the station self-reports). The self-reported fields (`spread_factor`, `mod_kind`, `pkt_type`) are also recorded for cross-checking. When they disagree, the script logs a warning but keeps the record — a disagreement indicates a firmware configuration error that is itself informative. Never discard mismatched records silently. `label_pkt` is set from the parsed `pkt_type` (the canonical authority for this label) rather than from the campaign config, because a single C5 run can produce DATA, ACK, and PAUSE rows whose ground-truth packet type can only be known after the row is captured.

4.4 Step 4 — Dry run (10-packet validation)

1. Verify Target A firmware has `LORA_SF 7` and `LORA_TX_PWR_DBM 14` (the defaults). Power on the Threat board and place it within radio range of Target A — the Threat forwards every intercepted packet automatically, no button press required. Target B off (no ACK responder).

2. **Identify the FPGA's USB-UART COM port.** The Nexys A7's onboard USB-UART bridge enumerates as a `USB Serial Device` (or `USB-Serial Port`) when the FPGA is powered and the bitstream is loaded. In Windows Device Manager (Ports section) or the Serial Monitor Logger's port list, identify the COM port that appears *only* when the Nexys is plugged in (unplug/replug to confirm). Note this port number — this is the port `collect.py` will read.

3. Run `python collect.py` from the `Code/Tools/CampaignCollect` directory. The script scans `campaigns/` for `*.json` files and displays a numbered menu with each config's `description` field.

4. Select the entry for `dry_run.json` (*DRY RUN — 10 packets, SF7, Target A*), enter the FPGA's USB-UART COM port when prompted, and confirm at the "Press Enter to begin collection" prompt. The output CSV is written automatically to `Code/Tools/CampaignCollect/logs/dry_run/results_dry_run.csv`; each subsequent run with this config overwrites that file.

5. Open `results_dry_run.csv` and verify: 10 rows, correct column names, `beacon_id = 1`, `spread_factor = 7`, `pkt_type = 0` (DATA) on every row, no CRC warnings in the terminal output.

6. If the script hangs at startup, no frames are reaching the host. Confirm the production bitstream is programmed (`UART_RXD_OUT` passthrough is part of `ingest_top.vhd`) and that the Nexys A7 USB-UART port is enumerating on the host PC. The Threat must be powered on with at least one Target transmitting.

4.5 Step 5 — Campaign C1: TX power sweep

1. Position Target A 3 m from the Threat antenna. Threat powered on and in range — no button press needed, it forwards all intercepted packets automatically. Target B off. C1 produces DATA-only rows from Beacon ID `0x01`.

2. TX power is controlled by `LORA_TX_PWR_DBM` at the top of the `/* USER CODE BEGIN PD */` block in `Core/Src/main.c` of the `Nucleo_RFM95_TargetA` project. Change only this one define for each sweep step:

Target TX power	Define to set
+2 dBm	`#define LORA_TX_PWR_DBM 2`
+8 dBm	`#define LORA_TX_PWR_DBM 8`
+14 dBm	`#define LORA_TX_PWR_DBM 14`
+20 dBm	`#define LORA_TX_PWR_DBM 20`

3. For each power level:
   - Update `LORA_TX_PWR_DBM` in `main.c`, press **F7** to build, then **F5** to flash and run.
   - Confirm on the USART2 debug terminal that the board prints `lora_init done: SF=7 TxPwr=`*N* `dBm` with the expected power level before collecting.
   - Run `python collect.py`, select the matching C1 entry from the menu (e.g. *C1 TX Power Sweep — +2 dBm*), and enter the FPGA's USB-UART COM port. Collect 200 packets.
   - Wait for the script to report "Done." Do not interrupt mid-run.

4. The four runs produce four separate CSV files in `Code/Tools/CampaignCollect/logs/`, one subfolder per config: `logs/campaign_c1_txpwr_2/results_campaign_c1_txpwr_2.csv`, `logs/campaign_c1_t` and so on for +14 and +20 dBm.

4.6 Step 6 — Validate Campaign C1

1. Concatenate the four CSV files and compute summary statistics (in Python with `pandas`):

```python
import pandas as pd
from pathlib import Path

logs = Path("logs")
files = [logs / f"campaign_c1_txpwr_{p}" /
         f"results_campaign_c1_txpwr_{p}.csv"
         for p in (2, 8, 14, 20)]
df = pd.concat([pd.read_csv(f) for f in files])
print(f"Total rows: {len(df)}")  # expect 800
print(df.groupby("run_id")["rssi_dbm"].agg(["mean","std","min","max"]))
```

Listing 3: Campaign C1 validation.

2. Verify: 800 total rows; mean RSSI increases (becomes less negative) as TX power increases; SNR positive at all power levels at 3 m distance.

3. Plot mean RSSI vs. TX power (scatter with error bars of $\pm 1\sigma$). The relationship should be monotonically increasing.

4.7 Step 7 — Campaign C2: Spreading factor sweep

1. Keep Target A at +14 dBm (`LORA_TX_PWR_DBM 14`), 3 m from the Threat antenna. Threat powered on, no button press needed. Target B off.

2. The spreading factor must be updated in **both** `Nucleo_RFM95_TargetA` and `Nucleo_RFM95_Threat` for each sweep step. The SX1276 receiver configures `RegModemConfig2` with a fixed SF and cannot auto-detect across different spreading factors in explicit-header mode — if Target A transmits at SF8 but the Threat is still configured for SF7, the Threat will not demodulate any packets. Set `LORA_SF` in `Core/Src/main.c` of *each* project to the same value before building:

Spreading factor	Target A define	Threat define
SF7	`#define LORA_SF 7`	`#define LORA_SF 7`
SF8	`#define LORA_SF 8`	`#define LORA_SF 8`
SF9	`#define LORA_SF 9`	`#define LORA_SF 9`
SF10	`#define LORA_SF 10`	`#define LORA_SF 10`

3. For each SF:

   - Update `LORA_SF` in `main.c` of **both** the Target A and Threat projects.
   - Press **F7** then **F5** in each VS Code window to build and flash both boards.
   - Confirm both debug terminals print `lora_init done: SF=`*N* `TxPwr=`*X* `dBm` with the matching SF before collecting.
   - Run `python collect.py`, select the matching C2 entry from the menu (e.g. *C2 SF Sweep — SF7*), and enter the FPGA's USB-UART COM port. Collect 200 packets.

4.8 Step 8 — Validate Campaign C2

1. Concatenate all six SF CSV files from their `logs/` subfolders:

```python
import pandas as pd
from pathlib import Path

logs = Path("logs")
files = [logs / f"campaign_c2_sf{sf}" /
         f"results_campaign_c2_sf{sf}.csv"
         for sf in range(7, 11)]
df = pd.concat([pd.read_csv(f) for f in files])
print(f"Total rows: {len(df)}")  # expect 800
```

Listing 4: Campaign C2 validation.

2. Verify 800 total rows and that `label_sf` cycles through values 7–10.

3. Compute `inter_arrival_ms` statistics per SF. The median inter-arrival time should increase with SF (each additional spreading factor roughly doubles the on-air time). Generate a box plot of `inter_arrival_ms` by `label_sf`.

[Figure placeholder]

`LH3F_c2_interarrival_boxplot.png`

Box plot of inter-arrival time (ms) by spreading factor from Campaign C2. Median inter-arrival time should approximately double with each SF increment, reflecting the increasing Radio Frequency (RF) on-air time.

Figure 1: Box plot of inter-arrival time (ms) by spreading factor from Campaign C2. Median inter-arrival time should approximately double with each SF increment, reflecting the increasing RF on-air time.

5 Verification Checklist

- ☐ Collection script handles CRC mismatches gracefully (logs warning, skips row, continues).
- ☐ Dry run: 10 rows, correct schema, no errors.
- ☐ C1: 800 total rows; mean RSSI monotonically increases with TX power.
- ☐ C2: 800 total rows; inter-arrival time increases with spreading factor across SF7–SF10.
- ☐ No sequence number gaps larger than 3 within a single campaign run (isolated gaps are acceptable; a run of gaps indicates a collection script crash or FPGA reset).

Lab Handout 3-F Deliverable

1. `collect.py` source code with inline documentation.
2. All campaign configuration JSON files: 9 total (1 dry-run + 4 for C1 TX-power levels + 4 for C2 spreading-factor levels).
3. Campaign C1 CSV files (4 files) and concatenated C1 summary: mean/std RSSI per power level, scatter plot of RSSI vs. TX power.
4. Campaign C2 CSV files (4 files) and box plot of inter-arrival time by spreading factor.
5. Preliminary data quality assessment: label mismatch count, CRC failure count, any anomalous runs.

Glossary

FPGA	Field-Programmable Gate Array	1
RF	Radio Frequency	11